Animus: An AI Exocortex with Production-Line Discipline

Author: ARETE · **Date:** 2026-06-02 · **Status:** Draft

> A single-engineer AI exocortex that applies Toyota Production System discipline, visible cost, quality gates, checkpoint/resume, and designed-in error-proofing, to autonomous AI agents. This paper documents what exists today (grounded in source, with honest maturity ratings), what it now makes possible, and where it should go next.

Feature maturity across documented subsystems: production: 68 · beta: 6 · experimental: 4 · stub: 5

Table of Contents

Animus: An AI Exocortex with Production-Line Discipline

Executive Summary

Animus is a personal AI exocortex: a single-engineer system that gives an AI agent durable memory, a stable identity, multi-agent coordination, and the ability to improve its own behavior, all running local-first on the operator's own

hardware. It is not a chat wrapper and not a framework chasing adoption. It is the daily tool one engineer built to think with, and its design treats an AI agent the way a manufacturing line treats a workstation: every unit of work has a visible cost, every defect stops the line, and nothing ships without passing a gate.

The core thesis is that the discipline missing from most agent systems is not intelligence, it's the Toyota Production System. LLM agents are expensive, forgetful, and helpful to a fault, and most frameworks treat cost and quality as observability afterthoughts you bolt on later. Animus inverts that. Token budgets are a hard execution constraint, not a dashboard: every LLM call routes through a `BudgetManager` that tracks per-agent and per-step spend against thresholds, and the budget gate is a constitutional non-negotiable, not a feature flag. Cost is made visible on one axis through an Effective-Tokens model (output weighted 4x, per-tier multipliers from Haiku at 0.08 to Opus at 5.0, Ollama at 0.0) so an output-heavy Opus workflow can't hide behind a low raw-token count. Quality gates halt or revise work between stages rather than letting a partial pass dilute into a passing average. And because a workflow persists per-stage checkpoints to SQLite, a run that fails at step four of six resumes at step four, not from the start.

The system is organized into four layers, each solving one problem. **Core** is identity and memory: a pluggable store (ChromaDB vector search fused with a BM25 keyword index via reciprocal rank fusion, with a JSON fallback) holding episodic, semantic, and procedural memories that are append-only, versioned with provenance lineage, and tier-classified for disclosure control. **Forge** is budget-gated orchestration and evaluation: a rubric-driven eval framework with LLM-judge metrics, dual failure taxonomies, and bootstrap-CI A/B comparison, plus three escalating self-improvement paths bounded by sandboxes and human approval gates. **Quorum** is supervisor-free coordination: agents negotiate shared intent through a stigmergic intent graph, a triumvirate voting engine, and pheromone-style markers, with no central controller. **Bootstrap** is the install-and-run daemon: a proactive scheduler that runs self-healing and nudge checks and drives the human-gated improvement loop.

The maturity picture is deliberately honest, because a whitepaper that overclaims is its own kind of defect. The memory layer, the budget and checkpoint machinery, the evaluation framework, the security hardening, and the Quorum coordination core are production code with tests behind them. The cross-device sync layer, the autoresearch evolution loop, and the Python self-improvement orchestrator are real but beta: built, wired, but not yet exercised under sustained load. A handful of advertised capabilities are spec-only and we say so plainly: the HOT/WARM/COLD tiered-memory engine is fully designed in docs but has zero implementing code; three of Quorum's four v2 observability features (active-inference resolver, liveness watchdog, coupling dashboard) are specs with no module behind them; and the Ogma synthesis persona exists only as a single verb on an unmerged branch.

Two claims that appear in older positioning documents do not survive contact with the code, and this paper retires them rather than repeat them. ARCHITECTURE.md describes Ed25519-signed memory nodes and AES-256 encryption at rest; the stores persist plaintext, and signing is unimplemented. A flagship "media engine producing roughly 480 videos a month" appears as a deployment proof-point; no such workflow exists in the repository. What Animus actually runs are developer and fleet-operations workflows, an evaluation harness used as a cross-project regression gate, and a self-improvement loop that turns observed tool failures into human-reviewed proposals. Those are smaller claims, and they are true.

What makes Animus worth describing is not any single layer but the through-line: a security and constitutional stance that treats the safe path as the default path, an evaluation framework that makes "better" a measured quantity rather than an opinion, and a memory model that never deletes. The rest of this paper walks each layer, names what is enforced versus aspirational, and shows where the manufacturing-line discipline holds and where it is still a goal.

1. Introduction and Positioning

An AI agent left running unattended has four failure modes, and they compound. It is **helpful to a fault**: asked to do a thing, it will do the thing, plus three things you didn't ask for, plus a confident summary of work it never actually performed. It is **expensive** in a way that hides: a single workflow can quietly spend an order of magnitude more than expected because output tokens cost roughly four times input tokens and a frontier model costs fifty times a small one, yet the raw token counter shows one undifferentiated number. It is **forgetful**: every session starts from amnesia, so the operator becomes the memory, re-explaining context that the system discarded the moment the window closed. And it is **unsafe to run unattended**: give it file I/O, web fetch, OAuth tokens, and a few message channels, and a prompt-injection payload or a mis-tagged secret has a clear path outward.

Existing agent frameworks address the orchestration mechanics but treat these four failures as someone else's problem. LangGraph and CrewAI give you graphs and roles; cost tracking is an integration you add, memory is a vector-store you wire up, and safety is your responsibility. Observability tools like LangSmith show you what happened after the money is spent. The pattern across the category is that budget, memory, coordination, and safety are afterthoughts layered onto an execution engine that was designed without them. You cannot retrofit a hard budget

ceiling into a system that assumed unbounded spend, and you cannot retrofit supervisor-free coordination into a message-passing topology that assumes a central orchestrator.

Animus's answer is to make those four concerns load-bearing from the foundation, and the organizing metaphor is a manufacturing line. On a Toyota line, cost is visible at every station, a defect stops the line rather than propagating, error-prevention is built into the fixture so the wrong part physically won't fit, and the standard of "good" is written down before the work starts. Animus applies the same posture to an agent. Spend routes through a single budget chokepoint that every LLM call must pass, so cost is visible and bounded by construction, not by audit. Memory is a first-class, append-only, versioned substrate that the agent reads and writes deliberately, so the system carries context the operator would otherwise hold. Coordination is stigmergic: agents leave and sense markers in a shared graph and resolve conflicts by evidence-weighted stability, so a swarm coordinates without a supervisor bottleneck. And safety is enforced in code at the egress boundary and below it at the kernel, with tier-scoped recall, DLP redaction at ingest, prompt-injection envelopes, integrity-checked boot, and systemd sandboxing, so the unsafe action is structurally blocked rather than merely discouraged.

The positioning that follows from this is narrow on purpose. Animus is a single-user personal exocortex, local-default with cloud-on-consent: the same model breadth as a cloud-routing agent via OpenRouter, but the opposite default, where PUBLIC-tier traffic may reach a cloud provider and PERSONAL, CONFIDENTIAL, and SECRET tiers fail closed. It is not a multi-tenant product, and a standing rule in the current roadmap explicitly resists adding SSO, RBAC, billing, or a landing page. The audience is one operator who wants an AI that remembers, costs what it says it costs, and can be left running overnight without becoming a liability. Everything in the architecture serves that operator, and the manufacturing-line discipline is how a single engineer keeps a system this broad from quietly rotting.

2. Design Philosophy

Four Toyota Production System concepts map directly onto Animus's architecture, and in each case the mapping is concrete code, not analogy.

Poka-yoke (error-proofing): make the wrong action structurally impossible, or at least structurally harder than the right one. The clearest example is the egress gate, a pure function every cloud-bound LLM call passes through: it denies CONFIDENTIAL and SECRET tiers unconditionally, denies PERSONAL under local-only mode, denies all non-loopback traffic when offline mode is set, and always permits loopback so a local Ollama model keeps working. The MCP server hard-pins memory recall to the PUBLIC tier, so confidential memories are structurally invisible to the automation surface even though the local user owns the underlying database. Recalled content is wrapped in an `<untrusted_data>` envelope that escapes nested close-tags so a crafted memory cannot break out and issue commands to the consuming model. Secret and PII redaction runs on every write, and the redaction record carries only a type and span, never the original value, so logging a hit cannot re-leak the secret. The self-audited length metric is poka-yoke applied to evaluation itself: it makes the model append its own claimed word count, then scores whether the claim matches reality, catching the confident-but-wrong self-report that a plain length check would miss.

We are honest about one place this principle is currently inverted. Tier-scoped recall accepts an `allowed_tiers` argument that defaults to `None`, meaning no filter, for backward compatibility. A caller who forgets to pass it leaks every tier. That is a poka-yoke failure: the safe path requires extra work and the unsafe path is the default. The fix (default-deny to PUBLIC) is identified and pending, and naming it here is itself part of the philosophy.

Jidoka (autonomation, "any agent may halt the line; never silently degrade"): stop and fix on defect rather than passing it downstream. Fail-fast gates in the evaluation framework force a FAILED status on any hard-gate metric scoring below 1.0 regardless of the composite, so "five of six structural gates pass equals a passing 0.83 average" cannot happen. Quality gates in the Core orchestrator halt execution or inject revision feedback back into the agent on failure. The tier router raises an error rather than silently falling back to cloud when no local provider is available for a sensitive tier: refusing to serve is the correct behavior, degrading quietly is not. The daemon refuses to boot if a SHA-256 integrity check of its critical-path files detects drift. The autoresearch evolution loop treats a missing or empty `better.md` as a hard stop, and pauses at 80 percent budget. The constitutional principle named Jidoka grants halt authority explicitly. The honest counterexample, again stated rather than hidden: LLM-judge metrics currently swallow exceptions and return a neutral 0.5 on any failure, which lets a broken judge or a provider outage masquerade as a mediocre-but-passing run. That violates jidoka, the failure taxonomy already has a `provider_error` bucket ready to receive it, and the fix is small and identified.

Standardized work: write down the definition of "good" before the work starts, and version it. Scoring rubrics are versioned YAML with named dimensions, weights, and a content hash for drift detection; three ship today (personal-quality, code-edit, briefing-quality). Prompts are versioned, diffable, content-hashed artifacts in a registry,

and every eval run is tagged with the prompt version that produced it, so a quality change can be attributed to a specific prompt revision. Four prompt-mode scaffolds (exploration, specification, evaluation, production) each enforce a single objective and a fixed output contract with explicit anti-blending rules, because a prompt that mixes exploration and specification and evaluation does all of them poorly. Thirty-plus declarative workflow definitions carry typed inputs, token budgets, and timeouts. The standard is authored by a human, and the system measures against it.

Kaizen (continuous improvement) with a human gate. Improvement is not autonomous-by-default; it is proposal-by-default. The self-healing check scans the last 24 hours of tool executions every six hours and turns failure patterns (a tool failing 30 percent of the time over five-plus runs, latency over ten seconds, or the same error three-plus times) into improvement proposals. The full self-improvement pipeline runs a thirteen-stage machine (analyze, plan, implement, test, apply, PR, merge) bounded by an allow-list, max-files and max-lines limits, an isolated sandbox, three human approval gates, and automatic rollback on test failure. The lighter-weight YAML workflow-evolution path is the constitutionally preferred channel for self-modification because changing a declarative YAML file is safer than changing Python. Auto-approval is hard-blocked in production unless an explicit environment flag is set. The A/B comparison closes the loop with rigor: it runs paired bootstrap resampling, default 1000 iterations, to put a 95 percent confidence interval on the score delta between two runs, and the CLI exits non-zero on a statistically significant regression. "Better" is a measured quantity with a confidence interval, not an opinion.

Underneath these four sits the **constitutional and safety stance**, which is what keeps an autonomous, self-modifying system bounded. Nine named principles (Sovereignty, Continuity, Transparency, Constraint, Proportionality, Budget Sovereignty, Arete, Jidoka, Subjectivity-Wins) are each annotated with the specific code construct that enforces them, and they are append-only, amendable only through a proposal manager with a human approval threshold. This doc-to-code annotation style, every principle pointing at its enforcing module, is the strongest pattern in the system and the template the rest of the documentation aspires to: it converts an ethics statement into a verifiable claim a reader can follow into the code. Memory is append-only and never deleted, even consolidation produces a new summary memory rather than mutating originals. A red-team driver uses a local uncensored model to generate adversarial probes across six attack categories against isolated fixtures, and project policy requires running it before merging any security-relevant change; it has already caught real bypasses (CamelCase and leetspeak credential markers), each of which became a regression test. The line that ties the philosophy together is that none of this is asserted as safe. It is enforced where the code enforces it, named where it does not yet, and measured wherever a measurement exists.

3. System Architecture

Animus is a single-operator AI exocortex organized as a monorepo of four packages, each carrying one responsibility. The design rule is that a layer solves exactly one problem and exposes a narrow contract to the layers above and below it. The four layers, from the substrate up:

Layer	Package	Responsibility
Core / Identity & Memory	`packages/core`	Persistence, self-knowledge, redaction, memory model, learning loop, identity self-model
Forge / Cognitive & Orchestration	`packages/forge`	Budget-gated agent orchestration, evaluation, self-improvement, providers
Quorum / Coordination	`packages/quorum`	Supervisor-free multi-agent coordination via a shared intent graph
Bootstrap / Interface & Daemon	`packages/bootstrap`	Install daemon, proactive scheduler, runtime wiring, the autonomous loop

The framing throughout is Toyota Production System discipline applied to running LLM agents: make cost visible, stop and fix on a defect rather than propagate it, and build error-prevention (poka-yoke) into the path rather than bolting it on as observability. This is not decoration. Three concrete invariants enforce it in code, and the rest of this document is honest about where the narrative outruns the implementation.

Control flow

A request enters either through the MCP server (a Claude Code session calling a memory tool over stdio) or through a workflow invoked on the Forge CLI. The MCP path is read-oriented: it resolves a memory tool, scopes recall to a sensitivity tier, redacts, wraps untrusted content in an injection envelope, records a metadata-only audit event, and

returns. The Forge path is execution-oriented: `WorkflowExecutor` loads a `WorkflowConfig` from YAML and drives steps sequentially, async-sequentially, or via an auto-parallel path. Every step passes through a budget gate before it runs and records usage after it completes. Approval steps halt the run, persist a resume token, and set the workflow to `awaiting_approval` so a human gate is a real stop, not a log line.

Data flow

Memory is the spine. Every `remember()` call runs redaction over content before persistence, so secrets and PII never enter the store in the first place. Reads flow back out through `recall()` / `search()` with an `allowed_tiers` filter that the ChromaDB backend pushes down into a `where` clause, gating what the automation surface can ever see. Cross-device sync computes JSON state deltas with SHA-256 checksums and a last-write-wins merge. The Forge layer threads a `CompletionRequest.sensitivity` field through a `TierRouter` that forces CONFIDENTIAL/SECRET traffic onto local Ollama and raises rather than silently falling back to cloud.

Budget flow

Budget is a first-class axis, not telemetry. Every LLM call in the orchestration and evolution paths routes through `BudgetManager`, which tracks per-agent and total usage against WARNING/CRITICAL/EXCEEDED thresholds with SQLite session persistence. It computes Effective-Tokens (ET = m × (1.0·input + 0.1·cache_read + 4.0·output)) with per-model-tier multipliers (haiku 0.08 through opus 5.0, ollama 0.0) so output-heavy and opus-tier spend is comparable to cheap local inference on one axis. This is the system's stated cost philosophy. It is honest to note here, and detailed in §4.5, that ET is currently a reporting metric: the live enforcement path still counts raw rolled-up tokens.

The three binding invariants

1. **Budget sovereignty.** Every LLM call routes through `BudgetManager`. This is a constitutional non-negotiable (Principle P6), enforced in both the evolution loop and the provider evaluator.
2. **Append-only, fail-closed memory.** Memory is never mutated in place; versioning creates a new record with a `parent_id`. Redaction runs on ingest. Tier gating runs on recall.
3. **Human-gated autonomy.** Self-modification routes through approval gates, sandboxed execution, and rollback checkpoints. The constitutionally-preferred path edits YAML workflows, not Python.

These invariants are the contract that translates the project's narrative into enforced behavior. Where they hold, the whitepaper makes verifiable claims. Where the narrative reaches past them, §4 says so plainly.

---

4. Subsystems in Detail

4.1 Core & Memory Layer

Purpose

The Core layer is Animus's persistence and self-knowledge foundation: a pluggable memory store holding episodic, semantic, procedural, and active memories with versioning, sensitivity tiers, and secret redaction; a transparent, approval-gated learning loop that mines patterns from stored memories; an identity self-model that lets Animus read its own codebase; and Ogma, a synthesis persona that reverse-engineers external work and internal subsystems into implementation-ready proposals.

Architecture

`MemoryLayer` (`memory/layer.py`) is a façade over a `MemoryStore` backend selected by a config string ("chroma" default, "local" JSON fallback), resolved through the package namespace so test mocks patch the looked-up name. Stores implement a `MemoryStore` ABC: store, update, retrieve, search, delete, list_all, get_all_tags. `ChromaMemoryStore` uses `chromadb.PersistentClient` with cosine HNSW and performs hybrid retrieval: a dense vector query fused with a lazily-rebuilt BM25Okapi keyword index via Reciprocal Rank Fusion (k=60), degrading to dense-only when `rank_bm25` is absent. `LocalMemoryStore` is a JSON file with atomic tmp-rename writes and naive substring search.

The `Memory` dataclass carries content, type, tags, confidence, source, and three layered field-sets: Context-Core versioning (version/parent_id/change_summary, with `get_version_history` walking the parent chain), provenance (direct/sync/consolidation/import/mcp), and Stage-2 Sensitivity re-exported from the shared `animus_types` package to avoid a Core→Forge dependency cycle. Every `remember()` runs `redact()` before persistence. The `LearningLayer` wires a `PatternDetector` (regex frequency/preference/correction/temporal extraction) into `LearnedItems` gated by a per-category approval table, with immutable guardrail checks, a transparency log, and rollback checkpoints.

Features

Feature	What it does	Maturity
Pluggable memory store	ChromaDB or JSON-fallback behind a `MemoryStore` ABC; import failure auto-falls-back	production
Hybrid dense+BM25 retrieval	Cosine vector search fused with BM25 via RRF (k=60); degrades to dense-only	production
Episodic/semantic/procedural/active model	`MemoryType` enum plus typed helpers (`remember_fact` , `remember_procedure` , `save_conversation`)	production
Secret/PII redaction on ingest	Two-tier regex on every `remember()` ; `RedactionHit` carries type/span/length, never the secret	production
Sensitivity disclosure tiers	Four tiers (PUBLIC/PERSONAL/CONFIDENTIAL/SECRET); `recall(allowed_tiers=...)` pushed into a ChromaDB `$in` clause	production
Versioning + provenance lineage	`update_with_version` never mutates in place; provenance tracks origin	production
Snapshot / restore / export / consolidation	Timestamped JSON snapshots, full restore, JSON/JSONL/CSV export, tag-grouped consolidation	production
Transparent, reversible learning loop	Pattern mining → approval-gated `LearnedItems` with guardrails, transparency log, rollback	production
Immutable guardrails	`GuardrailManager` checks learnings against immutable rules (no-exfiltrate, no-modify-guardrails) before apply	production
Identity self-model	`AnimusIdentity` resolves codebase root, exposes `read_own_file` / `list_own_files` , records reflections	production
Retrieval evaluation harness	Pure-functional Precision@k / Recall@k / MRR over a labeled fixture corpus, usable as a CI gate	production
Claude Code → ChromaDB sync tool	Parses CC memory files + notes repo, dedups by content hash, pushes with provenance=sync	production
Cross-device state sync	SHA-256-checksummed JSON deltas, last-write-wins merge; config sync strips api_key fields	beta
Ogma synthesis pipeline (read flow)	read→ground→prompt→parse into a frozen `OgmaOutput` ; no silent provider fallback on 401/403	experimental
Sensitivity backfill migration	One-shot tier classification queuing candidates to an audit JSONL for human review; never deletes	production

Honest maturity notes

Several headline capabilities are spec, not code, and the whitepaper names them as such:

- **HOT/WARM/COLD tiered memory** with promotion/demotion and lossless context compaction is fully specced in `docs/ANIMUS_MEMORY_GAPS.md` but has zero implementing code. The `Memory` dataclass has no tier, last_accessed, or access_count fields and no promotion job exists. This is the single largest doc-vs-code gap in the layer.
- **Ed25519-signed memory nodes and AES-256 encryption at rest**, claimed in `ARCHITECTURE.md` , are not implemented. Both stores persist plaintext.
- **Ogma** is unmerged on `feat/ogma-v0-read` ; the on-main `ogma/` directory is an empty stub and only the `read` verb exists.

Known scaling and correctness constraints, captured rather than hidden: `ChromaMemoryStore` mirrors every memory in a second in-memory dict, doubling RAM per record; the BM25 index re-tokenizes the entire corpus on any write (O(N) per write); `consolidate()` groups episodic memories by first tag and concatenates 150-char snippets with no LLM summarization; sync merge is naive last-write-wins by timestamp string with no vector clocks; and `allowed_tiers` defaults to `None` (no filter), making the tier gate opt-in rather than default-deny. That last point is a recognized poka-

yoke inversion: the safe path requires extra work, which contradicts the stated fail-closed posture, and the planned fix is to default to `{PUBLIC}`.

Integration

Core hoists the `Sensitivity` enum into `animus_types` so Forge providers can import it without a dependency cycle. `recall(allowed_tiers={PUBLIC})` is the read-time enforcement that gates the MCP server's egress. `EntityMemory.extract_and_link` wires memory content to a knowledge graph on every remember/import/consolidate. Ogma consumes Lugh `SourceItem`s, and both Ogma and the learning loop consume the cognitive layer with Ollama as the default local provider.

4.2 Forge, Autonomous Improvement & Evaluation

Purpose

Forge is the evaluation and autonomous-improvement layer. It provides a rubric-driven eval framework (suites, metrics, LLM-judge rubrics, dual failure taxonomies, a persistent run store, bootstrap-CI A/B comparison), three escalating self-improvement paths, and a library of prompt-mode scaffolds. The throughline is TPS discipline: a human authors the definition of "better" in the rubric, every LLM call passes the budget gate, and poka-yoke guardrails (fail-fast gates, max-iterations, drift checks, approval stages) bound the autonomy.

Architecture

Three layers under `packages/forge`. The eval framework in `evaluation/` defines `EvalCase` / `EvalResult` / `EvalSuite` / `EvalMetric` plus `AgentEvaluator` and `ProviderEvaluator`; `metrics.py` supplies 13 metric classes; `rubric.py` loads versioned YAML rubrics that reflectively instantiate metric classes by name and inject judge providers; `runner.py` executes suites sequentially or via a thread pool / asyncio semaphore; `store.py` persists runs; `compare.py` does paired/independent bootstrap resampling (default n=1000) for a 95% CI on the score delta and flags regressions. Two orthogonal classifiers tag results: technical (`FailureMode`) and content (F1–F8 `ContentFailureMode`).

The self-improvement layer offers three escalating paths: `workflow_evolution.py` is the YAML propose/approve/reject fast-path (the constitutionally-preferred safe channel); `evolution_loop.py` is a standalone autoresearch loop (hypothesis→experiment→evaluate→keep/discard, budget-gated, off by default); and `self_improve/orchestrator.py` drives a 13-stage `WorkflowStage` machine for actual Python changes, gated by `SafetyConfig` , `ApprovalGate` , `Sandbox` , and `RollbackManager` . The third layer is authoring assets: prompt-mode scaffolds, rubrics, eval suites, and 30+ workflow YAML definitions.

Features

Feature	What it does	Maturity
Versioned YAML rubrics	Reflective metric resolution by name, weighted composite + letter band, `content_hash` for drift; 3 ship (personal-quality, code-edit, briefing-quality)	production
Metric library (13 metrics)	ExactMatch, Contains, Regex, Similarity, LLMJudge, CodeExecution, Factuality, Safety, Length, SelfAuditedLength, NegativeExample, Composite	production
Self-audited length metric	Model appends `<wc>N</wc>` ; scores 1.0 honest, 0.3 dishonest divergence, 0.5 tag-absent, catches confidently-wrong self-counts	production
Fail-fast hard gates	`fail_fast: true` forces FAILED on any sub-1.0 score, preventing "5 of 6 gates pass = composite 0.83 = threshold met" dilution	production
Dual failure taxonomies	Technical (11 buckets on `failure_mode`) + content (F1–F8 list in metadata); one output can carry one technical + multiple content tags	production
Bootstrap-CI A/B comparison	Paired/independent resampling (n=1000), 95% CI on mean delta, regression flags; CLI exits code 2 on significant regression	production
Persistent run store + OutcomeTracker	Writes `eval_runs` / `eval_case_results` with cost and content tags; `feed_to_outcome_tracker` into the Forge tracker	production
Eval CLI	`run` / `list` / `compare` / `rubrics` / `results` ; live, `--mock` (no keys for CI), or `--adapter` to wire arbitrary agents	production
Prompt-mode scaffolds	exploration/specification/evaluation/production templates with fixed contracts and anti-blending rules	production

Feature	What it does	Maturity
Versioned prompt registry	`<name>@<version>` resolution with content hash for `eval_runs.prompt_version`, plus `diff()`	production
YAML workflow evolution fast-path	Validates a proposed YAML (version must increase), stages `.pending.yaml`, human approve/reject, append-only audit	production
Autoresearch evolution loop	Daemon-thread hypothesis loop gated by `better.md`, BudgetManager, max-iterations, identity-drift check; off by default	beta
Full self-improvement pipeline	13-state machine for Python changes: sandbox + three approval gates + auto-rollback on test failure	beta
Workflow definition library	30+ declarative multi-step YAML workflows with token budgets and typed I/O	production

Honest maturity notes

The eval framework is genuinely production-grade, but several sharp edges are real and named:

- **The content taxonomy (F1–F8) is built and persisted but never invoked in the live CLI run path.** `eval_cmd.py` calls only `tag_results` (technical), so `compare`'s F1–F8 delta tables are usually empty in practice. The fix is one call.
- **`CodeExecutionMetric` runs model-generated Python via bare `python` (not `sys.executable`, which the project's own rules ban) with only a timeout**, no real isolation despite the docstring claiming a sandbox. This is an RCE-shaped surface during evals.
- **LLM-judge metrics swallow all exceptions and return 0.5 on failure**, so a systematically broken judge silently scores everything mediocre-but-passing, masking provider outages.
- **The evolution loop's default experiment runner is a dry run** that echoes the plan string; without an injected runner, keep/discard verdicts are LLM opinion on a non-experiment. The loop is also not wired into any CLI, so the headline autoresearch capability is not operator-accessible out of the box.
- Residual `gorgon-` naming (branch prefix default, `gorgon-memory.db` / `gorgon-state.db`) marks incomplete Gorgon→Forge rename cleanup.

Integration

Every LLM call in the evolution loop and `ProviderEvaluator` routes through `BudgetManager` (constitutional P6). `ProviderEvaluator`, LLM-judge metrics, and the loop all call `Provider.complete(CompletionRequest)`; `MockProvider` serves CI. The evolution loop calls `IdentityAnchor.check_drift()` and loads constitutional principles P4/P5/P6/P8 into the hypothesis prompt as halt authority (Jidoka). The eval adapter mechanism wires external products (e.g. BenchGoblins `ClaudeService` via `--adapter module:callable`), making Forge eval a cross-project regression harness.

4.3 Quorum, Coordination & Active Inference

Purpose

Quorum (PyPI: `convergentAI`, import: `convergent`) is Animus's zero-dependency multi-agent coordination protocol. A swarm of agents negotiates shared intent without central control: each agent publishes `IntentNode`s describing what it provides and requires, a resolver detects overlaps and conflicts, a triumvirate voting engine resolves contested decisions, and a stigmergy field plus signal bus carry indirect coordination. This is the project's most-emphasized differentiator: agents coordinate through a shared SQLite-backed intent graph with stability scores and boid rules, never messaging a supervisor.

Architecture

The shipped core lives in `packages/quorum/python/convergent/` with an optional Rust PyO3 hot path. `GorgonBridge` composes the whole stack from one `CoordinationConfig`: `ScoreStore` + `PhiScorer` + `Triumvirate` (weighted voting with MAJORITY/UNANIMOUS/UNANIMOUS_HUMAN quorum levels and weighted-score tie-break), a `StigmergyField` (pheromone markers with exponential evaporation), a `FlockingCoordinator` (alignment/cohesion/separation over markers), and a `SignalBus` pub/sub with in-memory or SQLite cross-process backend. `IntentResolver` runs an intent against the graph via a `GraphBackend` protocol, producing Adjustments, ConflictReports, and adopted Constraints by structural overlap. Intent stability is a hardcoded weighted-sum over `EvidenceKind` tallies (base 0.3 plus capped bonuses for test passes, commits, consumed-by-other, and approval, minus conflicts and test fails). The Week-1 v2

addition is `EventLog`: an append-only bitemporal-lite SQLite timeline (valid_from world-time vs recorded_at observation-time, ported 1:1 from memboot) that mirrors events onto the SignalBus.

Features

Feature	What it does	Maturity
IntentResolver	Detects duplicate provisions, signature mismatches, conflicts by structural overlap; higher-stability intent wins provision rights	production
Evidence-weighted stability scoring	`compute_stability()` derives a 0–1 scalar from `EvidenceKind` tallies; mirrors the Rust `StabilityScorer`	production
Triumvirate voting engine	Consensus over 2–5 agents, configurable quorum level, weighted votes, tie-break, persisted vote records	production
Stigmergy field	Markers on files/tasks that decay exponentially and reinforce on repeat; SQLite-persisted	production
FlockingCoordinator	Style/pattern constraints (alignment), task-drift (cohesion), overlap flags (separation) from markers	production
Signal bus	Pub/sub transport with consumer tracking; SQLite backend for cross-process coordination	production
GorgonBridge facade	Single entrypoint composing the full stack; `enrich_prompt`, `request_consensus`, `submit_agent_vote`, `evaluate`	production
Coordination EventLog (v2 Week 1)	Append-only bitemporal-lite timeline with correlation IDs, range queries, best-effort SignalBus mirror	production
Consciousness-Quorum bridge	Forge thread runs budget-gated reflection over logs + intents + P1–P9, publishes insights as stability-0.3 intents; 39 tests	production
Active-inference IntentResolver (v2 Week 3–4)	Bayesian-posterior stability + curiosity score from variance	stub (spec only)
LivenessWatchdog (v2 Week 2)	Stalled/dead detection over the event stream with warn/stalled/dead ladder + Discord alerts	stub (spec only)
Coupling MI dashboard (v2 Week 5)	Read-only sliding-window mutual-information heatmap over the marker stream	stub (spec only)

Honest maturity notes

This is where the whitepaper must be most careful. **Three of the four advertised v2 capabilities, active-inference resolver, LivenessWatchdog, coupling dashboard, are specs only with zero code.** The planned paths (`resolver/active_inference.py`, `liveness/`, `coupling/`) do not exist. Any claim of active inference or liveness monitoring as a working feature would be false.

Further grounding:

- Stability scoring is a brittle hardcoded weighted-sum that the active-inference spec itself flags as saturating: 100 confirmations of a contested thing score the same as 100 of an obvious thing. The pluggable `StabilityScorer` protocol refactor that would fix this is not yet done.
- The EventLog landed at the convergent root, not the roadmap's content-addressed core path with `prior_state_hash` / `new_state_hash`. It is an audit log, not the deterministic transition log the roadmap promised, so the "bisect which mutation broke a stable intent" replay use case is not achievable from the current schema.
- EventLog wiring is opt-in (`event_log=None` default), and nothing enforces that a bridge wires one in; a bridge built without it produces an empty timeline.
- Coordination `*.db` files are committed into the repo, mixing runtime state into version control.

Integration

Forge depends on Quorum (`convergentai ^1.1.0`); `GorgonBridge` enriches agent prompts and runs consensus voting. The Consciousness-Quorum bridge lives Forge-side, gates every reflection call through `BudgetManager.can_allocate()` under `agent_id='consciousness_bridge'`, loads constitutional P1–P9 into the reflection prompt, and degrades gracefully when convergent is absent (`HAS_CONVERGENT` guard). EventLog's bitemporal pattern is a direct port of memboot's locked design, and the planned coupling tile targets the localhost:7700 bootstrap dashboard.

4.4 Security & Hardening

Purpose

A defense-in-depth layer protecting the tiered memory store and bounding autonomous agent behavior. It enforces a local-first / cloud-on-consent posture through code-level egress gates, DLP redaction, tier-scoped recall, prompt-injection envelopes, integrity/tamper detection, systemd kernel sandboxing, and an LLM-driven red-team driver, applying TPS jidoka ("any agent may halt the line; never silently degrade") to AI safety.

Architecture

The hardening is a "7-track" stack documented in `docs/THREAT_MODEL.md` and verified by `verify_hardening.py` (25 wired scenarios). Two enforcement planes exist. The **application plane** runs in-process: redaction at ingest and MCP egress, tier-scoped recall pinned to `{PUBLIC}` in the MCP server, prompt-injection envelopes with close-tag escaping, a metadata-only audit log, the egress chokepoint (`is_egress_allowed`), and Forge-side provider egress re-checks plus a `TierRouter` where CONFIDENTIAL/SECRET force local Ollama and raise rather than fall back to cloud. The **kernel plane** is systemd unit hardening: `ProtectSystem=strict` , `ReadOnlyPaths=/home/arete` with explicit carve-outs, `NoNewPrivileges` , `PrivateTmp` , and `IPAddressDeny` for all RFC1918/link-local/ULA ranges with loopback-only allow. Tamper detection runs at daemon boot: `verify_or_raise()` SHA-256s a curated critical-path file set against an on-disk baseline and refuses to boot on drift.

Features

Feature	What it does	Maturity
Tier-aware egress gate	Pure-function chokepoint: denies non-loopback under `ANIMUS_OFFLINE` , denies CONFIDENTIAL/SECRET unconditionally, denies PERSONAL under `ANIMUS_LOCAL_ONLY` , always allows loopback	production
DLP credential/PII redaction	Universal token patterns (OpenAI/Anthropic/GitHub/AWS/Stripe/Slack/private-key/SSN/bearer) + env-configurable personal patterns; hits carry no original value	production
Red-team-hardened detectors	CamelCase + leetspeak credential patterns added reactively after the driver caught Qwen-generated bypasses, each with a regression test	production
Tier-scoped MCP recall	All three MCP memory tools hard-pin scope to `{PUBLIC}` , structurally hiding confidential/ secret memories from automation	production
Prompt-injection envelope	Wraps recalled chunks in `<untrusted_data>` with a "reference not commands" footer; escapes nested close-tags to prevent breakout	production
SHA-256 integrity baseline	Daemon refuses to boot on critical-path drift unless `ANIMUS_INTEGRITY_OVERRIDE=1`	production
Append-only metadata-only audit log	Daily-rotated JSONL recording tool name, tier scope, counts, byte length, never raw content; `record()` never raises	production
systemd kernel sandboxing	Kernel-layer network filter and filesystem lockdown a code bypass cannot defeat	production
Red-team driver	Local uncensored Ollama generates adversarial probes across 6 categories against isolated fixtures; exits non-zero on findings ≥ threshold	production
Deterministic hardening verification	25 end-to-end scenarios across real components, pytest-mirrored as a CI gate	production
OpenRouter provider (PUBLIC-only + open-weights)	Fails closed on non-PUBLIC; rejects closed-vendor model slugs at config-load and per-request; never auto-default	production
TierRouter sensitivity override	CONFIDENTIAL/SECRET force local; absence of a local provider raises `ProviderError` , never silent cloud fallback	production
Constitutional principles (P1–P9)	Nine principles each mapped to enforcing code; amendable only via a 20%-threshold proposal manager with human approval	production
Forge field-level encryption	Fernet authenticated encryption keyed via PBKDF2-HMAC-SHA256, plus brute-force lockout and request-size limits	production
Forge self-improve sandbox	Path allow-list, gitleaks pre-commit scan, strict default-branch mode against the whip-saw vector	production

Honest maturity notes

The hardening is the most mature subsystem, but its threat model is explicit about its own boundaries, and the whitepaper inherits that honesty:

- **Tier enforcement is not a defense against a malicious local shell user**, the adversary IS the `arete` account and can read ChromaDB directly. **Disk encryption at rest is documented as not in place** (plain ext4, no LUKS), so laptop theft yields full plaintext exposure of all classified memories. This is the single largest unmitigated real-world threat.
- The egress gate **trusts the caller-supplied tier and does not inspect content**, so a CONFIDENTIAL memory mis-tagged PUBLIC would egress.
- The integrity baseline tracks only 4 files; the tier-router, OpenRouter provider, `pi_wrap`, and the integrity checker itself are not in the set, so tampering with those bypasses boot-time detection.
- **Two separate egress-policy implementations** (Core and Forge) are hand-synced; the Forge module's own docstring admits "drift is detectable by inspection." A shared canonical module in `animus_types` would remove this class of silent drift.
- The prompt-injection envelope's effectiveness depends on the consuming model honoring the footer, and is tested only against Claude Sonnet, unverified against Qwen/Llama.
- systemd unit files live in a user-writable directory outside the read-only mount and are not versioned in the repo, so an adversary editing the unit defeats the kernel-plane filter.
- The unified secret manager in `SECURITY_LAYER.md` (age/pass/1Password backends, `secrets://` URI resolution) is largely an unbuilt spec; only Forge's Fernet `FieldEncryptor` exists, and it uses a hardcoded legacy `gorgon-` PBKDF2 salt when unset.

Integration

Redaction runs at `MemoryLayer` ingest; tier-scoped recall is the read-time enforcement of the Sensitivity schema. The MCP server is the egress surface where every call passes scope-pin → redact → wrap → audit. `CompletionRequest.sensitivity` threads through `TierRouter` and every cloud provider's `_check_request_egress`, with OpenRouter the strictest adopter. The Bootstrap daemon calls `verify_or_raise()` at startup. Ollama is both the trusted local provider for sensitive traffic and the engine for the red-team driver's adversarial generation. External scanners (gitleaks, CodeQL, bandit, pip-audit) run pre-commit and in CI.

4.5 Orchestration & Proactive Engine

Purpose

The orchestration layer coordinates multi-agent workflows under hard token-budget and cost discipline, persisting state to SQLite so a workflow that fails mid-run resumes at the failed step rather than from the start. The proactive engine is a separate daemon-resident scheduler in Bootstrap that runs periodic self-healing and nudge checks, feeding a human-gated autonomous self-improvement loop. Together they apply TPS ideas, make cost visible, stop-and-fix on defect, build error-prevention into the path, to running LLM agents.

Architecture

Two distinct orchestration engines exist. The **production engine** in `packages/forge` is `WorkflowExecutor`, a mixin-composed class (12 handler mixins, 20+ step types) that loads a `WorkflowConfig` from YAML and drives steps sequentially, async-sequentially, or auto-parallel. Each step passes `_check_budget_exceeded` (can_allocate + a daily-limit query) before running and records usage afterward into the BudgetManager, FeedbackEngine, ExecutionManager, and a task-history store. `CheckpointManager` / `StatePersistence` persist per-stage checkpoints to SQLite; a failure at step 4 of 6 restarts at step 4. The **embedded engine** in `packages/core/animus/forge` is a lighter `ForgeEngine` that runs agents in order, enforces budgets, and evaluates `GateConfig` quality gates via a safe no-eval parser with halt/revise semantics, this is what `bootstrap_loop.py` uses for the Core→Forge→Quorum two-agent self-review cycle.

The proactive subsystem in `packages/bootstrap` is a `ProactiveEngine` owning an asyncio scheduler loop that computes next-fire times (cron or `every Nm` interval), fires checks, logs nudges to WAL SQLite, gates delivery on quiet hours, and spawns observer tasks to capture implicit outcomes. `AnimusRuntime` boots components in fixed order, registers six builtin checks, and injects deps into the `self_heal` check. The autonomous loop closes when `self_heal` (every 6h) turns tool-failure patterns into proposals, which `self_improve` tools apply through an `ImprovementSandbox` (backup +

hot-reload) under impact scoring with rollback, while larger Python changes route to the full `SelfImproveOrchestrator` with sandbox + three approval gates + PR.

Features

Feature	What it does	Maturity
Token budget manager + Effective-Tokens	Per-agent/total usage, thresholds, reserve, SQLite persistence; ET model (4× output, haiku 0.08…opus 5.0, ollama 0.0)	production
Pre-flight budget validation	Estimates workflow cost from per-agent profiles + prompt length; PASS/WARN/FAIL against available budget; strict mode fails under 25% margin	production
Checkpoint / resume to SQLite	Per-stage checkpoints (input/output/tokens/duration/status); `resume_from` maps a step id to a start index	production
Mixin-composed executor with budget enforcement	12 mixins, 20+ step types; budget check before each step, usage recorded after	production
In-workflow approval gates	`approval` step halts, persists a resume token + next-step id, sets `awaiting_approval`, the real wired quality gate	production
Quality gates in the embedded Core orchestrator	`GateConfig` evaluated by a hand-written safe parser (no eval) with halt/revise; backs the bootstrap self-review loop	production
Proactive engine	Asyncio scheduler, quiet hours, WAL nudge log with self-migrating prediction columns, post-delivery outcome observers	production
Six builtin checks incl. self-heal	morning_brief, task_nudge, calendar, reflection (3 AM → LEARNED.md), verdict_sync, self_heal (every 6h; proposes on ≥30% failure rate / >10s latency / repeated error)	production
Human-gated self-improvement orchestrator	10-stage analyze→sandbox→PR with three approval gates; auto_approve hard-blocked unless `ANIMUS_FORGE_ALLOW_AUTO_APPROVE=1`	production
Impact measurement + rollback	Baseline vs post on failure_rate (60) and latency (40) → −100..+100 score; rollback restores from per-proposal backup + hot-reload	production
Bootstrap runtime orchestrator	Boots components in fixed order, wires self-heal deps and observers, `reload_config()` hot-updates quiet hours + sandbox config	production
YAML workflow evolution fast-path	`.pending.yaml` staging with single approve/reject gate; validator presented as doc-illustrative, not a located module	experimental

Honest maturity notes

This subsystem has two consequential gaps between the README narrative and the production code:

- **The README/quickstart YAML `gates:` block is not parsed by the production Forge loader.** `WorkflowConfig` has no `gates` field; quality-gate enforcement exists only in the separate Core embedded engine. The production executor's sole gating primitive is the `approval` step type. This is a credibility gap for a whitepaper-referenced repo and should be reconciled by either implementing parsing or removing the example.
- **Effective-Tokens is reporting-only, not enforced.** `_check_budget_exceeded` / `record_usage` still use raw rolled-up tokens, so a budget can read "OK" while real cost is several times higher. Making ET the enforced unit is the highest-leverage refinement here.
- `BudgetManager.allocate()` does not reserve a pending allocation, it returns `True` after `can_allocate` without recording, so concurrent auto-parallel steps can each pass the check and collectively overspend.
- `self_heal` dedup uses a process-lifetime module-global set, so a daemon restart can re-propose the same failing tool.
- `_compute_impact_score` weighs only failure_rate and latency, with no signal for a correctness/quality regression, so a change that speeds things up while degrading output scores positively.

Integration

`bootstrap_loop` builds a two-agent self-review workflow and reaches Quorum consensus before writing improvements. `WorkflowExecutor` feeds per-step outcomes into a `FeedbackEngine` and task-history store used by analytics. `self_heal` and `self_improve` call a cognitive backend (Anthropic/Ollama/Forge) for root-cause analysis; provider tier alters plan aggressiveness. State persists across `budget_session_usage`, workflow checkpoints, `approval_tokens`, `proactive_log`,

`proactive_outcomes`, and improvement proposals. Identity-file changes route through `IdentityProposalManager` rather than direct writes.

4.6 Vision, Use-Cases & Stated Roadmap

Purpose

This subsystem is the narrative and intent layer: the positioning documents, philosophy, use-case catalog, case study, competitive analysis, and the multiple stated roadmaps. It frames Animus as a "personal AI exocortex for cognitive sovereignty", a single-engineer system applying TPS discipline to multi-agent AI. Its job is to map why Animus is built and for whom, not how the code works. It is included in this document because a whitepaper-grade system must be honest about the distance between its narrative and its code, and this subsystem is precisely where that distance lives.

Features

Feature	What it does	Maturity
Four-layer architecture narrative	Core/Forge/Quorum/Bootstrap, each solving one problem; all four map to real packages	production
TPS / lean-manufacturing framing	Budget-first, make-cost-visible, Jidoka/Poka-yoke/Andon, grounded in the wired BudgetManager	production
Stigmergic coordination pitch	Shared intent graph + stability scores + boid rules, no supervisor; backed by Quorum code	production
Constitutional principles (P1–P9)	Each principle annotated with enforcing code, the strongest doc-to-code grounding in the subsystem	production
Local-default / cloud-on-consent positioning	Backed by the recently-shipped OpenRouter PUBLIC-only + open-weights enforcement	production
Competitive landscape analysis	Honest market map (CrewAI, OpenClaw, Hermes Agent, LangSmith) with candid overlap admissions	production
Use-case catalog	30 personal-assistant scenarios, explicitly aspirational, with an anti-patterns section	experimental
Phased implementation roadmap (Phase 0–6)	Phases 0–4 largely complete; Phase 5 self-learning and Phase 6 wearable remain aspirational	beta
Personal roadmap (single-user doctrine)	Most current, load-bearing roadmap (2026-05-15); re-grounds to "best tool for ONE user" with a resist-productization checklist	production
Research-assistant roadmap (RA-0… RA-4)	Re-scopes Animus as a pure-local research assistant; RA-0 locked, RA-1+ to be written	beta
Quorum v2 roadmap	Disciplined 5-week plan that explicitly rejects most of an external spec, roadmap-as-filter	beta
Case study (portfolio artifact)	Interview-oriented; notably stale metrics (333 tests / 18K LOC) contradicting the README's 13,676 tests	experimental
Developer-tools / Arete-Tools ecosystem vision	PyPI tools + 9-tool suite with revenue tiers; mostly spec, superseded by the anti-productization roadmap	stub
Media Engine flagship deployment (~480 videos/mo)	Three autonomous YouTube channels in 8 languages, the whitepaper's headline proof-point	stub

Honest maturity notes

This subsystem carries the largest vision-vs-reality gaps in the entire project, and naming them is the point:

- **The flagship deployment is fictional in-repo.** The whitepaper's ~480-videos/month Media Engine and 5-platform Marketing Engine, its primary "this architecture handles real workloads" proof, have zero implementing code. A grep finds only vendored `googleapiclient` stubs and an unrelated Lugh daily-digest. The actually-exercised workloads are developer and fleet-ops oriented (code-review, fleet-incident-triage, security-audit, bounty-watcher). This is the biggest integrity risk for a technical reader who greps, and the whitepaper should re-frame this section as "specced but not built" or replace it with the real workloads.

- **The documents contradict each other on maturity and metrics.** `CASE_STUDY.md` claims Phase 5 "complete" and 333 tests / 18K LOC; `ROADMAP.md` shows Phase 5 fully unchecked; the README reports 13,676 tests. The case study predates build-out and was never updated.
- **Strategic whiplash is undocumented.** The whitepaper and landscape doc sell a commercial gateway/PyPI product strategy with $8–49/mo tiers; the Personal Roadmap explicitly forbids productization (no SSO/RBAC/billing/landing-page). A reader handed both has no signpost that the latter supersedes the former.
- Four parallel roadmaps exist with no authoritative index of which is canonical and which is historical. The recommended fix is a one-page "Document Status & Canon" index tagging each doc CANONICAL / HISTORICAL / SUPERSEDED-BY with dates.
- The 9-tool Arete-Tools suite and Phase 6 hardware (wearable ring, vehicle mode, portable-storage) are pure aspiration with no code or hardware path.

Integration

Where the narrative is grounded, it is grounded well: the TPS/budget claim maps to the wired `BudgetManager`; the stigmergy pitch maps to the Quorum intent graph; the self-improvement claim maps to `self_improve/orchestrator.py` and `evolution_loop.py`; and each constitutional principle is annotated with its enforcing module. The `CONSTITUTIONAL_PRINCIPLES.md` doc-to-code annotation style, every principle pointing at the construct that enforces it, is the strongest grounding pattern in the project and is the template every vision claim in this document aspires to follow.

5. Possibilities

This section is the practical answer to a fair question: now that Core, Forge, Quorum, Security, and the Proactive Engine exist as working code, what can Animus actually do that a stock LLM chat client or a framework like LangGraph cannot? The scenarios below are grounded in shipped subsystems, not roadmap intent. Where a scenario leans on a part that isn't fully built, it's marked.

5.1 Compounding memory that survives the model

A conversation in ChatGPT is amnesiac by design: the context window is the memory, and it evaporates. Animus inverts that. Every `remember()` call persists through a redaction poka-yoke, lands in a hybrid dense+BM25 store with Reciprocal Rank Fusion retrieval, carries a sensitivity tier and a provenance tag, and is versioned rather than overwritten. The store already holds roughly 1,400 classified memories.

What this enables concretely: ARETE can ask "what did we decide about the egress gate, and why" and get back the actual decision plus its lineage, because `update_with_version` keeps the parent chain instead of mutating in place. The Claude Code sync tool means memory accrues passively. Every session's learnings flow into the same ChromaDB through `animus_sync.py` with content-hash dedup, so the exocortex grows without a dedicated capture ritual. The value compounds: the corpus that answers today's question was assembled by hundreds of prior sessions that never had to be told to write things down.

The differentiator over "RAG over my notes" is the discipline around the store: append-only writes, redaction on ingest so secrets never enter the index, tier gating so a recall can be scoped to PUBLIC before it crosses an automation boundary, and a retrieval eval harness (Precision@k / Recall@k / MRR) that can gate retrieval quality in CI. The memory isn't just persistent. It's measurable and auditable.

5.2 Safe unattended operation

The combination that makes autonomy tolerable for a single operator is the budget gate plus checkpoint/resume plus the security planes. A workflow that fails at step 4 of 6 resumes at step 4, not from scratch, because `CheckpointManager` persists per-stage state to SQLite and the executor resolves a resume index. Every step passes a budget check before it runs. And the whole process runs under a systemd unit with `ProtectSystem=strict`, `ReadOnlyPaths=/home/arete`, and `IPAddressDeny` for all RFC1918/link-local ranges, so a code-level bypass still hits a kernel-layer wall.

The realistic scenario this unlocks is the overnight delegate: hand Animus a research question before bed, and it works through a checkpointed task queue under a hard cost ceiling, with CONFIDENTIAL/SECRET tiers structurally

barred from leaving the machine. The pieces for this exist today in isolation (checkpoint/resume, budget enforcement, TierRouter no-cloud-fallback, the proactive scheduler). The end-to-end "wake to a sourced briefing" loop is RA-3 roadmap, not shipped. What is shipped is the safety substrate that makes running it unattended a reasonable thing to do rather than a reckless one.

5.3 A self-improvement loop with human-held brakes

Animus can watch itself fail and propose its own fixes. The `self_heal` proactive check runs every six hours, scans the last 24 hours of tool-execution history, and auto-creates improvement proposals when a tool crosses a failure-rate, latency, or repeated-error threshold. Those proposals route through a sandbox: apply on a branch, run tests, snapshot, and either promote to a draft PR behind three human approval gates or auto-rollback on test failure. Impact is measured (failure rate weighted 60, latency 40) so a change that doesn't help gets backed out.

The honest framing: this is a Kaizen loop with the brakes held by a human, which is exactly the point. `auto_approve` is hard-blocked in production unless an explicit environment flag is set. The YAML workflow-evolution fast-path is the constitutionally preferred channel for self-modification because it touches declarative config, not Python. The system is built so the cheap, reversible, low-blast-radius changes are easy and the dangerous ones require a person. This is the TPS Jidoka principle made literal: any gate may halt the line, and nothing degrades silently into production.

5.4 Supervisor-free multi-agent coordination

Quorum lets a small swarm of agents (5 to 20, by design) negotiate shared intent without a central orchestrator. Agents publish IntentNodes describing what they provide and require; the resolver detects overlap and conflict structurally; a triumvirate voting engine settles contested decisions; and a stigmergy field carries indirect coordination through decaying pheromone markers. Higher-stability intent wins provision rights, so the swarm self-organizes around evidence rather than a designated leader.

This is the project's hardest-to-retrofit differentiator. The coordination state is a shared SQLite intent graph, not a message bus, which means it's queryable and auditable after the fact via the append-only EventLog. The realistic use today is internal: the Consciousness-Quorum bridge runs a budget-gated reflection pass over monitoring logs and open intents and publishes insights back as low-stability intents, giving the system a structured way to surface its own tensions. The active-inference rescoring that would make stability scores robust to evidence flooding is specced but not built, so the coordination is real while the "principled curiosity signal" on top of it is still ahead.

---

6. Design Refinement

These are the highest-leverage improvements to systems that already exist. The bar here is not "build something new" but "make the shipped thing match its own contract." Items are grouped by theme and prioritized by a blend of security/correctness impact and effort. The top of the table is where a reader who can do exactly one thing should start.

Priority table

#	Theme	Current weakness	Refinement	Effort	Priority
1	Memory / Safety	`allowed_tiers` defaults to `None` (no filter), so a caller that forgets the argument leaks all tiers. The safe path requires extra work; the unsafe path is the default.	Make tier filtering default-deny: require an explicit tier set or default to `{PUBLIC}`, so a forgotten argument fails closed.	medium	P0
2	Eval	The content failure taxonomy (F1-F8) is fully built and persisted but never called in the live eval path. `compare`'s content-delta tables are dead UI.	Wire `tag_content_failures` into `eval_cmd.py` run alongside `tag_results`, gated on the applied rubric's dims.	small	P0
3	Eval / Safety	`CodeExecutionMetric` runs model-generated Python via subprocess with bare `python` (banned by project rule) and only a timeout: no	Switch to `sys.executable` and reuse the `self_improve` Sandbox or a real restricted namespace.	medium	P0

#	Theme	Current weakness	Refinement	Effort	Priority
		isolation despite the docstring claiming a sandbox.			
4	Security	Two egress-policy implementations (Core and Forge) are hand-synced; the Forge copy's own docstring admits "keep this in sync." Drift silently weakens one plane.	Collapse both into a single implementation in the shared `animus_types` package and import from both sides.	small	P0
5	Orchestration	`BudgetManager.allocate()` checks but does not reserve, so parallel steps can each pass `can_allocate` and collectively overspend.	Implement real reservation: track pending allocations, decrement on record or release.	medium	P1
6	Orchestration	Effective-Tokens (the stated cost model: 4x output, tier multipliers) is computed for reporting but enforcement still counts raw tokens. A budget can read "OK" at 5x real cost.	Route `record_usage` and `_check_budget_exceeded` through `effective_tokens` with the model id.	medium	P1
7	Eval	LLM-judge metrics swallow all exceptions and return 0.5, so a broken judge masquerades as mediocre-but-passing and corrupts composites and CIs.	Return an ERROR status or a `judge_error` flag instead of a silent 0.5; route into the existing `provider_error` bucket.	small	P1
8	Security	The integrity baseline hashes only 4 files. The tier-router, OpenRouter provider, pi_wrap, and the checker itself are untracked, so tampering with them passes boot detection.	Expand the tracked set to include router, providers, pi_wrap, both egress copies, and a self-hash; store a detached signature.	medium	P1
9	Coordination	EventLog wiring is opt-in (`event_log=None` default); a bridge built without one produces an empty timeline, undermining the audit guarantee the log exists to provide.	Enforce EventLog presence at the `GorgonBridge` composition layer so all mutation sites emit by construction.	small	P1
10	Coordination	Stability scoring is a brittle hardcoded weighted-sum vulnerable to evidence flooding; the pluggable-scorer refactor is the prerequisite for active inference.	Extract the `StabilityScorer` protocol and have `compute_stability()` delegate to an injectable scorer now, ahead of the AI work.	medium	P2
11	Memory	BM25 re-tokenizes the entire corpus on every write (`_bm25_dirty`), O(N) per write on a 1,400+ store.	Use incremental add/remove (rank_bm25 supports appending) or cache the tokenized corpus keyed by memory id.	medium	P2
12	Memory	`ChromaMemoryStore` mirrors every memory in an in-memory dict, doubling RAM and capping scale.	Hydrate Memory objects from ChromaDB on demand; keep only an id-to-metadata index for BM25.	large	P2
13	Orchestration / Eval	Quality-gate YAML in the README quickstart is not parsed by the production loader; gates only exist in the separate Core engine. The headline feature doesn't exist where the docs say.	Either implement `gates` parsing in the production `WorkflowConfig` (reusing Core's safe parser) or remove the example from the README.	medium	P2
14	Security	`field_encryption.py` uses a hardcoded default PBKDF2 salt (`gorgon-field-encryption-v1`), weakening key derivation against precomputation.	Generate a per-install random salt persisted to `~/.config/animus` ; drop the legacy Gorgon constant.	small	P2
15	Hygiene	Residual Gorgon naming (`branch_prefix` default, `gorgon-*.db` artifacts, committed coordination `*.db` files in version control) and stale docs (OpenRouter spec header still says "DRAFT, not approved to build").	Finish the rename; gitignore runtime SQLite; correct the doc headers.	small	P3

Reading the table

The P0 band shares a single property: every item is a place where the safe behavior is the harder path or where a claimed guarantee isn't actually wired in. Item 1 is the canonical poka-yoke inversion: a security gate that's opt-in is not a gate. Items 2 and 3 are eval-integrity issues where the code exists but isn't called, or executes untrusted code without the isolation it advertises. Item 4 removes an entire class of silent drift from a security-load-bearing function. None of the P0 items is large. They are cheap precisely because the hard work was already done and only the last wire is missing.

The P1 band is correctness under load: budget reservation, enforced cost weighting, judge-failure visibility, and broadened tamper detection. These matter the moment the system runs unattended or in parallel, which is the direction the roadmap points.

P2 and P3 are scaling ceilings and doc-versus-code honesty: real, worth doing, but not the things that would bite first.

---

7. Future Work

Refinement makes the existing system trustworthy. This section is net-new capability. The items below are ranked by impact (transformative / significant / incremental) and then sequenced into phases, because some of them are prerequisites for others and building them out of order wastes effort.

7.1 Ranked candidates

Transformative

- **Research-assistant capability layer (RA-1/RA-2).** WebFetch with an allowlist, Retrieve over the corpus, Cite, Synthesize, with source-grounded retrieval as the default output contract ("answers without sources" = failure). This is the stated canonical direction and the point at which Animus becomes ARETE's daily tool rather than a thing he builds.
- **Auto-promotion eval loop.** When `eval compare` shows a statistically significant improvement (CI excludes zero, positive) for a new prompt version, automatically open a WorkflowEvolution pending patch pinning that version. This turns the eval suite into the experiment runner the evolution loop is currently missing, closing the Kaizen loop within the existing approval gates.
- **HOT/WARM/COLD tiered memory with lossless compaction.** Fully specced, zero implementing code today. It's the single largest doc-versus-code gap in the memory layer and the differentiator the competitive analysis leans on. Real tiers would materially change retrieval quality and context-window economics.
- **Replay/bisect over a content-addressed coordination log.** Add `prior_state_hash` / `new_state_hash` to the EventLog and build a replay engine that reconstructs graph state, enabling deterministic bisection of which mutation destabilized a stable intent.

Significant

- **Active-inference IntentResolver.** Replace the hardcoded weighted-sum stability with a surprise-weighted Bayesian posterior and a `curiosity_score` from posterior variance, then wire curiosity into Forge's self-improvement loop to direct work toward under-evidenced intents. This is the one behavior change both the Quorum-v2 and Personal roadmaps commit to.
- **Overnight delegate (RA-3).** Persistent SQLite task queue, turn-level checkpoint/resume, and a morning digest of task → outcome → cost → citations, with a measured one-week unattended intervention rate. This exercises the checkpoint/resume and budget claims under real autonomous load: the validation the (unbuilt) Media Engine was supposed to provide.
- **Encryption at rest plus signed memory records.** Close the gap between ARCHITECTURE.md's stated Ed25519/AES-256 guarantees and a reality of plaintext ext4. Laptop theft currently yields full plaintext exposure of all classified memories. This is the largest unmitigated real-world threat for an exocortex carried on a laptop.
- **Durability/rebuild and plain-text export.** A timed cold-VM Bootstrap dry-run plus an `animus export --all` portable-archive command with a stable, documented schema. For a single-user system holding years of state, loss-of-machine is the catastrophic failure mode.
- **Judge-calibration / meta-eval harness.** Periodically score the LLM judges against a human-labeled golden set and track per-model judge drift as a first-class metric. All quality scoring rests on judges whose reliability is currently assumed, not measured.
- **Unified secret manager.** The age/pass backend, `animus secrets` CLI, and `secrets://` URI resolution are fully specced with build prompts but unbuilt. Credential handling stays ad-hoc, and that ad-hoc handling is the documented source of repeated in-session key leaks.

Incremental

- **LivenessWatchdog over the event stream** (warn/stalled/dead ladder, fleet-monitor Discord alerts). Nothing currently watches individual coordination steps; the event log already provides the substrate.

- **Coupling MI dashboard** over the stigmergy marker stream, scoped as observability-only.
- **Cost/quality Pareto optimizer** over (model, prompt_version, rubric) using stored run history to recommend the cheapest config holding a target band. The data already exists in the run store.
- **Power/sample-size advisor on `compare`**, so a "not significant" verdict on a 10-case suite isn't misread as "no difference."

7.2 Sequencing

The temptation is to build the exciting things first (active inference, tiered memory). The opinionated sequence inverts that, because resilience and the eval loop are prerequisites and the flashy items are leaf nodes.

Phase A, Earn the right to run unattended. Durability/rebuild dry-run, plain-text export, and encryption at rest. These are cheap insurance against the one failure mode (lost machine, corrupted state) that erases everything else. Do them before the system holds more state worth losing. Pair with the P0 refinements from Section 6, since default-deny tiers and a unified egress module are part of the same "trustworthy substrate" investment.

Phase B, Close the Kaizen loop. Auto-promotion eval loop plus judge-calibration. The auto-promotion loop is what makes self-improvement genuinely autonomous within the approval gates, but it's only safe if the judges it depends on are calibrated. Build them together, calibration slightly ahead. This is also where the StabilityScorer protocol extraction (refinement #10) pays off, since it's the prerequisite for active inference in Phase D.

Phase C, Make it the daily tool. RA-1/RA-2 research capability layer, then the RA-3 overnight delegate. This is the canonical direction and the moment Claude Code reverts to advisory. The delegate validates the autonomy claims under real load, which is the evidence the project's positioning has been missing.

Phase D, The differentiators on top. Tiered memory, active-inference IntentResolver, and the replay/bisect engine. Each is high-impact but each depends on earlier work: tiered memory wants the durability story settled, active inference wants the scorer protocol and a calibrated eval harness to prove it actually improves behavior, and replay wants the content-addressed event log. These are the headline capabilities, and they're deliberately last because building them on an unproven substrate produces impressive demos that can't be trusted.

Continuous, Observability leaf nodes. LivenessWatchdog, coupling dashboard, Pareto optimizer, and the power advisor can be slotted in opportunistically; none blocks anything and each is low-risk, high-signal. The roadmap's own stop-and-measure gate applies: build one, confirm it earns its keep against a real workload, then decide whether to deepen it.

---

8. Open Questions and Risks

A whitepaper that only lists strengths is marketing. These are the genuine unknowns, stated plainly, because a reader evaluating Animus (or ARETE) should weigh them honestly.

8.1 Where do the safety boundaries actually hold?

The security model is explicit that it does not defend against a malicious local shell user: the adversary the threat model assumes away *is* the `arete` account, which can read ChromaDB directly. Disk encryption at rest is documented as not in place, so laptop theft means full plaintext exposure of roughly 1,400 classified memories. The egress gate trusts the caller-supplied tier and doesn't inspect content, so a memory mistakenly tagged PUBLIC would egress. And the prompt-injection envelope's effectiveness ultimately depends on the consuming model honoring a footer instruction, which has been tested against Claude Sonnet but not against the Qwen/Llama open-weight models that the local-first posture pushes toward becoming the primary consumers. The honest position: the boundaries that are code-enforced (tier routing, egress gate, kernel sandbox) are strong; the boundaries that depend on model compliance or on the operator's own disk hygiene are assumptions, and they're documented as such rather than papered over.

8.2 Is a single maintainer sustainable?

Animus is one engineer's system, and that's both its coherence and its fragility. The Gorgon-to-Forge rename is still incomplete in code, four parallel roadmaps contradict each other with no canonical index, and the flagship "480 videos/month" deployment in the whitepaper has zero implementing code. None of these is fatal, but together they signal the characteristic risk of a solo project: documentation drifts from reality faster than one person can reconcile it, and the parts that aren't load-bearing for daily use rot first. The Personal Roadmap's "resist productization" turn is the right instinct here. Scoping to one user is the only way a single maintainer keeps the surface area honest. The open question is whether the discipline holds, or whether the next exciting idea reopens the productization sprawl the roadmap just closed.

8.3 Are the evals actually valid?

Every quality judgment in Forge ultimately rests on LLM-judge metrics, and their reliability is assumed rather than measured. The failure taxonomy names `judge_disagreement` but nothing currently quantifies judge accuracy against a human-labeled set. Worse, judge failures currently return a silent 0.5 (refinement #7), so a systematically broken judge looks like a mediocre-but-passing run. The bootstrap-CI comparison machinery is statistically sound, but it can only be as trustworthy as the scores feeding it, and on small suites a "no significant difference" verdict may simply be underpowered. Until judge calibration exists, the eval framework should be read as a strong *relative* signal (is version B better than version A on the same judge) rather than a trustworthy *absolute* one.

8.4 What happens to cost at scale?

The Effective-Tokens model is the right idea, but it's currently reporting-only: enforcement counts raw tokens, so the budget can read healthy while real spend is several times higher on output-heavy or opus-tier work. The budget reservation gap means parallel steps can collectively overspend before any of them records usage. And two divergent cost tables (a stale 2024 pricing table versus the tier-multiplier model) produce inconsistent numbers between dashboards and budgets. For a system whose entire philosophical pitch is "make cost visible," these are the gaps most worth closing, because the cost discipline is the claim the TPS framing rests on. None is hard to fix (they're in the Section 6 P1 band), but until they are, the cost-sovereignty guarantee is aspirational at the margins where it matters most: unattended, parallel, output-heavy runs.

8.5 The honest summary

Animus is further along than most solo AI projects and more honest about its gaps than most funded ones. The substrate (memory, budget, security planes, checkpoint/resume, coordination) is real and substantial. The autonomy is gated the right way. The largest risks are not architectural but operational: documentation drift, unmeasured eval validity, at-rest encryption, and the question of whether one person can sustain the reconciliation work that keeps the narrative matching the code. Those are the right risks to have, because every one of them is closeable with the refinements and future work already on the table.

Appendix A: Subsystem Evidence Index

Every feature claim in this paper is grounded in source. This index lists each subsystem's features, honest maturity rating, and the file evidence the analysts cited.

Core & Memory Layer

The Core & Memory Layer is Animus's persistence and self-knowledge foundation: a pluggable memory store (ChromaDB vector + JSON fallback) holding episodic/semantic/procedural/active memories with versioning, sensitivity tiers, and secret redaction; a transparent, reversible learning loop that mines patterns from stored memories into approval-gated "learned items"; an identity self-model that lets Animus read its own codebase; and Ogma, a synthesis persona that reverse-engineers external work and internal subsystems into implementation-ready proposals. It applies TPS-style discipline, append-only memory (never delete), approval gates by category, rollback checkpoints, redaction poka-yoke, and a retrieval eval harness as a metric gate.

Feature	Maturity	Evidence
Pluggable memory store (ChromaDB + JSON fallback)	production	`packages/core/animus/memory/layer.py:55-62; packages/core/animus/memory/stores/base.py:10`
Hybrid dense+BM25 retrieval with RRF fusion	production	`packages/core/animus/memory/stores/chroma.py:144-280; packages/core/animus/memory/fusion.py:10-27`
Episodic/semantic/procedural/active memory model	production	`packages/core/animus/memory/types.py:17-23,157-211; packages/core/animus/memory/layer.py:143-216,483-502`
Secret/PII redaction on ingest	production	`packages/core/animus/memory/redaction.py:38-185; packages/core/animus/memory/layer.py:101-110`
Sensitivity disclosure tiers + read-side gating	production	`packages/core/animus/memory/layer.py:226-256; packages/core/animus/memory/stores/chroma.py:248-250; packages/core/animus/memory/types.py:34-39`
Memory versioning + provenance lineage	production	`packages/core/animus/memory/layer.py:313-401,591-609`
Snapshot / restore / export / consolidation	production	`packages/core/animus/memory/layer.py:403-467,506-572,611-691`
Transparent, reversible learning loop	production	`packages/core/animus/learning/__init__.py:150-367; packages/core/animus/learning/patterns.py:168-451; packages/core/animus/learning/categories.py:35-42`
Immutable guardrails on learning	production	`packages/core/animus/learning/guardrails.py:21-118; packages/core/animus/learning/__init__.py:177-189`
Identity self-model / code self-reference	production	`packages/core/animus/identity.py:21-129`
Retrieval evaluation harness	production	`packages/core/animus/memory/evaluation.py:112-234`
Claude Code → ChromaDB memory sync tool	production	`packages/core/animus/../../../tools/animus_sync.py:46-65,430-537`
Entity extraction + memory linking	production	`packages/core/animus/memory/layer.py:130-139,475-480; packages/core/animus/entities.py:207,517,701`
Sensitivity backfill migration	production	`packages/core/animus/scripts/migrate_sensitivity_tiers.py:1-40`
Cross-device state sync (delta/merge)	beta	`packages/core/animus/sync/state.py:158-444; packages/core/animus/protocols/sync.py:11-26`
Ogma synthesis pipeline (read flow)	experimental	`feat/ogma-v0-read:packages/core/animus/ogma/read.py:1-30; feat/ogma-v0-read:packages/core/animus/ogma/models.py:30-80`

Observed limitations: HOT/WARM/COLD tiered memory with promotion/demotion and lossless context compaction is fully specced in docs/ANIMUS_MEMORY_GAPS.md but has NO implementing code, the Memory dataclass has no tier/last_accessed/access_count fields, and no promotion job exists. Pure aspirational spec.; ARCHITECTURE.md claims Ed25519-signed memory nodes and AES-256 encryption at rest; the code has no signing of Memory records and no encryption in either store (ChromaDB and JSON persist plaintext content). The signature field in the gap-spec metadata schema is unimplemented.; Ogma is unmerged on feat/ogma-v0-read and the on-main packages/core/animus/ogma/ directory is an empty pycache stub; only the read verb exists (brief/gap/audit/sweep verbs from OGMA.md are not in code).; ChromaMemoryStore holds a complete second copy of every memory in an in-memory dict (self._memories) for metadata and BM25, duplicates ChromaDB's own storage and will not scale memory-wise to very large corpora.; BM25 index rebuilds the entire corpus from scratch on any store/delete (_bm25_dirty flag), O(N) re-tokenization on every write; no incremental update.; consolidate() groups episodic memories only by their first tag and concatenates 150-char snippets with no LLM summarization, crude and lossy; not the 'summary block' described in the compaction spec.; Pattern detection is purely regex/heuristic (PREFERENCE_INDICATORS, hour-bucketing) with no semantic/LLM understanding; brittle to phrasing and prone to shallow 'frequently requests' matches.; Sync merge is naive last-write-wins by updated_at string comparison with no vector clocks or conflict surfacing; concurrent edits on two devices silently drop the loser. apply_delta only inspects top-level keys 'memories'/'learnings'/'guardrails', so per-record adds nested under those keys are the only path exercised.; allowed_tiers defaults to None (no filter) for backward compatibility, so any caller that forgets to pass it leaks all tiers, the gate is opt-in, not default-deny.; recall_by_tags and get_memory(partial-id) call store.list_all() and scan in Python, full table scans that bypass the

vector index.; Redaction is regex-only: novel credential formats or non-pattern PII (names, addresses in prose) pass through unredacted; acknowledged in the red-team comments as an ongoing cat-and-mouse.

Forge, Autonomous Improvement & Evaluation

The evaluation + autonomous-improvement layer of Animus Forge. It provides a rubric-driven eval framework (suites, metrics, LLM-judge rubrics, dual failure taxonomies, persistent run store, bootstrap-CI A/B compare), three escalating self-improvement paths (YAML workflow evolution fast-path, an autoresearch-style evolution loop, and a full sandbox+approval+rollback Python self-improvement pipeline), and a library of prompt-mode scaffolds. The throughline is Toyota Production System discipline: a human authors the definition of "better"/the rubric, every LLM call passes the budget gate, and poka-yoke guardrails (fail-fast gates, max-iterations, drift checks, approval stages) bound the autonomy.

Feature	Maturity	Evidence
Versioned YAML scoring rubrics with reflective metric resolution	production	`packages/forge/src/animus_forge/evaluation/rubric.py:140 (build_metrics); packages/forge/rubrics/personal-quality.yaml`
Metric library (13 metrics: deterministic + LLM-judge)	production	`packages/forge/src/animus_forge/evaluation/metrics.py:210 (LLMJudgeMetric), metrics.py:561 (SelfAuditedLengthMetric)`
Self-audited length metric (poka-yoke for word-count honesty)	production	`packages/forge/src/animus_forge/evaluation/metrics.py:561-637`
Fail-fast hard gates on metrics	production	`packages/forge/src/animus_forge/evaluation/base.py:142,358,377; packages/forge/eval_suites/benchgoblins-ask.yaml`
Dual orthogonal failure taxonomies (technical + content)	production	`packages/forge/src/animus_forge/evaluation/failure_taxonomy.py:62; packages/forge/src/animus_forge/evaluation/failure_taxonomy_content.py:77`
Bootstrap-CI A/B run comparison with regression gating	production	`packages/forge/src/animus_forge/evaluation/compare.py:88 (bootstrap_ci_delta), compare.py:176 (compare_runs); eval_cmd.py:315`
Persistent eval run store + OutcomeTracker feed	production	`packages/forge/src/animus_forge/evaluation/store.py:38 (record_run), :274 (feed_to_outcome_tracker); eval_cmd.py:202-219`
Eval CLI (run/list/compare/rubrics/results)	production	`packages/forge/src/animus_forge/cli/commands/eval_cmd.py:15-225`
Prompt-mode scaffolds (exploration/specification/evaluation/production)	production	`packages/forge/prompts/modes/README.md; prompts/modes/exploration.md; prompts/modes/evaluation.md`
Versioned prompt registry	production	`packages/forge/src/animus_forge/evaluation/prompt_registry.py:1-40,202 (diff)`
YAML workflow evolution fast-path	production	`packages/forge/src/animus_forge/coordination/workflow_evolution.py:85,112,158; cli/commands/evolve.py`
Workflow definition library (30+ YAML workflows)	production	`packages/forge/workflows/refactor.yaml:1-35; 30+ files in packages/forge/workflows/`
Autoresearch evolution loop	beta	`packages/forge/src/animus_forge/coordination/evolution_loop.py:116,285 (_iterate),:441 (_load_better)`
Full self-improvement pipeline (sandbox + approval + rollback) for Python changes	beta	`packages/forge/src/animus_forge/self_improve/orchestrator.py:30-60 (WorkflowStage); self_improve/safety.py:20 (SafetyConfig); cli/commands/self_improve.py`

Observed limitations: Content failure taxonomy (F1-F8) is built and persisted by the store but NOT wired into the eval CLI run path, eval_cmd.py only calls tag_results (technical), never tag_content_failures. Content tags are populated only if a caller invokes the classifier manually, so compare's content_failure_delta tables will usually be empty in practice.; CodeExecutionMetric runs extracted Python via subprocess with bare 'python' (not sys.executable, which CLAUDE.md explicitly bans) and only a timeout, no real sandbox/isolation despite the docstring saying 'in a sandbox'. Arbitrary model-generated code executes on the host.; LLM-judge metrics swallow all exceptions and return 0.5 on any failure (no provider, parse failure, API error). A systematically broken judge silently scores everything 0.5 rather than erroring, which can mask provider outages as mediocre-but-passing runs.; The evolution loop's default experiment runner is a dry run that just echoes the plan string ('[dry run] Plan executed: {plan}'), without an injected experiment_runner, the loop generates and 'evaluates' hypotheses that were never actually tested, so keep/discard verdicts are LLM opinion on a non-experiment.; EvolutionLoop is standalone, not wired into any CLI command, supervisor, or daemon (only referenced by its own module + tests). It can only be driven programmatically, so the

documented autoresearch capability is not operator-accessible out of the box.; SafetyConfig.SafetyMetric unsafe-pattern list is a tiny hardcoded regex set (3 patterns: bomb/weapon, harm instructions, kill/hurt someone), trivially bypassable and not a serious content-safety gate without the optional safety_provider.; The eval base computes the composite as an unweighted mean of metric scores (sum/len); only the Rubric path applies dim weights. A suite run without --rubric ignores any intended metric weighting.; branch_prefix in SafetyConfig still defaults to the legacy 'gorgon-self-improve/' name, indicating incomplete rename cleanup from the Gorgon->Forge migration (also gorgon-memory.db / gorgon-state.db artifacts in the package root).; FailureClassifier judge-disagreement and flaky detection depend on case.metadata fields (judge_samples, flaky) that nothing in the run path populates, these buckets can only fire on hand-labeled fixtures.

Quorum, Coordination & Active Inference

Quorum (PyPI: convergentAI, import: convergent) is Animus's zero-dependency multi-agent coordination protocol library. It lets a swarm of agents negotiate shared intent without central control: each agent publishes IntentNodes describing what it provides/requires, a resolver detects overlaps/conflicts/constraints, a triumvirate voting engine resolves contested decisions, and a stigmergy field plus signal bus carry indirect coordination. A v2 extension layer adds observability (event log, planned liveness watchdog, coupling dashboard) and a planned active-inference rescoring of intent stability; only the event log has landed so far. The Consciousness-Quorum bridge (built in Forge) wires the reflection loop into this graph as low-stability intents.

Feature	Maturity	Evidence
IntentResolver (overlap/ conflict/constraint resolution)	production	`packages/quorum/python/convergent/resolver.py:110-260`
Evidence-weighted stability scoring	production	`packages/quorum/python/convergent/intent.py:172-194`
Triumvirate voting engine	production	`packages/quorum/python/convergent/triumvirate.py:36,287,323`
Stigmergy field (pheromone coordination)	production	`packages/quorum/python/convergent/stigmergy.py:49,215`
FlockingCoordinator (alignment/cohesion/ separation)	production	`packages/quorum/python/convergent/flocking.py:132,149,179,211`
Signal bus (pub/sub, in-memory + SQLite cross-process)	production	`packages/quorum/python/convergent/signal_bus.py:44; packages/quorum/python/convergent/sqlite_signal_backend.py:44`
GorgonBridge integration facade	production	`packages/quorum/python/convergent/gorgon_bridge.py:37-286`
Coordination EventLog (Quorum v2 Week 1)	production	`packages/quorum/python/convergent/event_log.py:113; resolver.py:54-70; tests/test_mutation_sites_emit.py`
Consciousness-Quorum bridge	production	`packages/forge/src/animus_forge/coordination/consciousness_bridge.py: 130-417; cli/commands/consciousness.py; tests/test_consciousness_bridge.py`
Active-inference IntentResolver (Quorum v2 Week 3-4)	stub	`docs/specs/quorum_v2_week3-4_active_inference_resolver.md (Status: Ready to build); no matching code under packages/quorum`
LivenessWatchdog (Quorum v2 Week 2)	stub	`docs/specs/quorum_v2_week2_liveness_watchdog.md (Status: Ready to build); packages/quorum/liveness MISSING`
Coupling MI dashboard (Quorum v2 Week 5)	stub	`docs/specs/quorum_v2_week5_coupling_dashboard.md (Status: Ready to build); packages/quorum/coupling MISSING`

Observed limitations: Three of the four advertised v2 capabilities, active-inference resolver, LivenessWatchdog, coupling dashboard, are specs only with zero code. The planned paths (packages/quorum/resolver/ active_inference.py, packages/quorum/liveness/, packages/quorum/coupling/) do not exist. Any whitepaper claim of active inference or liveness monitoring as a working feature would be false.; Stability scoring is a brittle hardcoded weighted-sum over evidence tallies (intent.py:172), which the active-inference spec itself flags as saturating and vulnerable to evidence flooding (100 confirmations of a contested thing score the same as 100 of an obvious thing). The StabilityScorer protocol refactor that would make it pluggable has not been done.; The EventLog landed at the Python convergent root, NOT at the roadmap's specified content-addressed core path (packages/core/ontology/ events/tick_event.py with prior_state_hash/new_state_hash). It is an event audit log, not the content-addressed transition log with deterministic state hashes the roadmap promised, the 'bisect which mutation broke a stable intent'

replay use case is not actually achievable from the current schema.; EventLog wiring is opt-in (event_log=None default on PythonGraphBackend, Triumvirate, StigmergyField); if a caller constructs these without passing an EventLog, no events are emitted and there is no enforcement that the bridge wires one in.; The SignalBus mirror on EventLog is best-effort with a blanket except Exception swallow (event_log.py:281), so observability silently degrades on subscriber errors with only a warning log.; FlockingCoordinator cohesion/separation are keyword/marker heuristics (_extract_keywords, flocking.py:294), not semantic, drift detection is shallow.; predict_trajectories returns {} unless a semantic_matcher (LLM) is configured (resolver.py:432), so trajectory prediction is inert in the default zero-dependency configuration.; The consciousness bridge reflection parser falls back to a regex {…} scrape and returns an empty 'Parse failure' output on malformed JSON (consciousness_bridge.py:362-372); a misbehaving local model silently produces no coordination signal for that cycle.; Coordination DBs are committed into the repo (convergent_coordination*.db), mixing runtime state into version control.

Security & Hardening

A defense-in-depth layer that protects Animus's tiered memory store (PUBLIC/PERSONAL/CONFIDENTIAL/SECRET) and bounds autonomous agent behavior. It enforces a local-first / cloud-on-consent posture through code-level egress gates, DLP credential redaction, tier-scoped memory recall, prompt-injection envelopes, integrity/tamper detection, systemd kernel sandboxing, and an LLM-driven red-team driver, applying TPS jidoka ("any agent may halt the line; never silently degrade") to AI safety.

Feature	Maturity	Evidence
Tier-aware egress gate (is_egress_allowed)	production	`packages/core/animus/network/egress.py:62; packages/forge/src/animus_forge/network/egress.py:44`
DLP credential/PII redaction	production	`packages/core/animus/memory/redaction.py:38-72,123`
Red-team-hardened detectors (CamelCase + leetspeak)	production	`packages/core/animus/memory/redaction.py:57-72; docs/THREAT_MODEL.md:284-296`
Tier-scoped MCP recall (scope pin)	production	`packages/core/animus/mcp_server.py:143-145,192,332`
Prompt-injection envelope ()	production	`packages/core/animus/mcp_server.py:72-84; packages/forge/src/animus_forge/security/pi_wrap.py:30-44`
SHA-256 integrity baseline / boot tamper detection	production	`packages/core/animus/integrity/checker.py:36-41,145; packages/bootstrap/src/animus_bootstrap/daemon/__main__.py:21-52`
Append-only metadata-only audit log	production	`packages/core/animus/audit/egress_log.py:53-78,81`
systemd kernel sandboxing	production	`~/.config/systemd/user/animus.service and animus-forge.service (IPAddressDeny/ProtectSystem/ReadOnlyPaths lines verified)`
Red-team driver (LLM-generated adversarial probes)	production	`packages/core/animus/redteam/driver.py:116-364,570-575; docs/THREAT_MODEL.md:277-296`
Deterministic hardening verification suite	production	`packages/core/animus/scripts/verify_hardening.py:547-629`
OpenRouter provider with PUBLIC-only egress + open-weights guarantee	production	`packages/forge/src/animus_forge/providers/openrouter_provider.py:140-194; packages/forge/src/animus_forge/tui/providers.py:61-68`
TierRouter sensitivity override + no-cloud-fallback	production	`packages/forge/src/animus_forge/providers/router.py:244-267`
Constitutional principles (P1-P9)	production	`docs/CONSTITUTIONAL_PRINCIPLES.md:18-62`
Forge field-level encryption + abuse controls	production	`packages/forge/src/animus_forge/security/field_encryption.py:1-30; security/{brute_force,request_limits,audit_log}.py`
Forge self-improve sandbox (allow-list + gitleaks + whip-saw)	production	`packages/forge/src/animus_forge/self_improve/safety.py and pr_manager.py (exercised by verify_hardening.py:229-296)`

Observed limitations: Two separate egress-policy implementations (Core animus/network/egress.py and Forge animus_forge/network/egress.py) are manually kept in sync by hand, the Forge module's own docstring says 'Keep this in sync ... drift is detectable by inspection.' A divergence would silently weaken one plane.; The integrity baseline tracks only 4 files (redaction.py, egress.py, mcp_server.py, audit/egress_log.py). The tier-router, OpenRouter provider, pi_wrap, and integrity checker itself are NOT in the tracked set, so tampering with those bypasses boot-time detection.; Tier enforcement is explicitly NOT a defense against a malicious local shell user, the adversary IS the arete account and can read ChromaDB directly (THREAT_MODEL.md A2). Disk encryption at rest is documented as NOT IN

PLACE (plain ext4, no LUKS), so laptop theft = full plaintext exposure of ~1423 memories.; The egress gate trusts the caller-supplied tier and does not inspect content (redteam _probe_egress_smuggle notes it 'doesn't inspect content, it trusts the tier on the request'); a mis-tagged CONFIDENTIAL memory marked PUBLIC would egress.; The prompt-injection envelope's effectiveness ultimately depends on the consuming model honoring the footer; THREAT_MODEL.md assumption 5 notes it is tested only against Claude Sonnet, unverified against Qwen/Llama.; systemd unit files live in ~/.config/systemd/user/ which is itself writable by the user and outside the ReadOnlyPaths mount (THREAT_MODEL.md assumption 3), an adversary editing the unit defeats the kernel-plane network filter. The hardening directives are also NOT versioned in the repo, only on the operator's disk (with .bak copies).; The secret-manager design in docs/SECURITY_LAYER.md (age/pass/1Password backends, animus secrets CLI, secrets:// URI resolution) is largely an UNBUILT spec with embedded build prompts, no shared/secrets/ module or animus.secrets package was found; only Forge's Fernet FieldEncryptor exists.; field_encryption.py uses a hardcoded default PBKDF2 salt ('gorgon-field-encryption-v1', legacy Gorgon naming) when ENCRYPTION_SALT is unset, weakening key derivation against precomputation if the secret_key is also weak.; OpenRouter's open-weights guarantee uses a hardcoded denylist of 6 closed-vendor prefixes; a new closed vendor namespace on OpenRouter would pass until the list is updated (acknowledged in code comments).; In-memory / RAM-only subversion and root-level gdb memory dumps are explicitly out of scope with no mitigation (THREAT_MODEL.md). The OpenRouter spec doc header still says 'DRAFT, parked, not approved to build' despite the provider being fully implemented and registered, doc/code drift.

Orchestration & Proactive Engine

The orchestration layer coordinates multi-agent AI workflows under hard token-budget and cost discipline, persisting state to SQLite so a workflow that fails mid-run resumes at the failed step rather than from the start. The proactive engine is a separate, daemon-resident scheduler (in Bootstrap) that runs periodic self-healing and nudge checks, feeding a human-gated autonomous self-improvement loop. Together they apply Toyota Production System ideas, make cost visible, stop-and-fix on defect, build error-prevention into the path, to running LLM agents.

Feature	Maturity	Evidence
Token budget manager with Effective-Tokens cost model	production	`packages/forge/src/animus_forge/budget/manager.py:96 (effective_tokens), :132 (BudgetManager), :280 (record_usage), :165 (_restore_from_db)`
Pre-flight budget validation	production	`packages/forge/src/animus_forge/budget/preflight.py:257 (validate), :140 (DEFAULT_ESTIMATES)`
Checkpoint / resume to SQLite	production	`packages/forge/src/animus_forge/state/checkpoint.py:222 (resume), packages/forge/src/animus_forge/workflow/executor_core.py:189 (_find_resume_index), :473 (start_workflow wiring)`
Mixin-composed workflow executor with budget enforcement	production	`packages/forge/src/animus_forge/workflow/executor_core.py:33 (class), :206 (_check_budget_exceeded), :522 (_execute_sequential)`
In-workflow approval gates (halt + persisted resume token)	production	`packages/forge/src/animus_forge/workflow/executor_core.py:664 (_handle_approval_halt), :357 (awaiting_approval finalize)`
Quality gates in the embedded Core orchestrator	production	`packages/core/animus/forge/gates.py:28 (evaluate_gate), packages/core/animus/forge/engine.py:133 (_evaluate_gates), packages/core/animus/forge/models.py:45 (GateConfig)`
Proactive engine (scheduler + nudges + outcome calibration)	production	`packages/bootstrap/src/animus_bootstrap/intelligence/proactive/engine.py:76 (class), :285 (_scheduler_loop), :384 (_observe_window)`
Six built-in proactive checks including self-heal	production	`packages/bootstrap/src/animus_bootstrap/intelligence/proactive/checks/__init__.py:18 (get_builtin_checks), packages/bootstrap/src/animus_bootstrap/intelligence/proactive/checks/self_heal.py:47 (_run_self_heal), :205 (every-6h schedule)`
Human-gated self-improvement orchestrator (analyze→sandbox→PR)	production	`packages/forge/src/animus_forge/self_improve/orchestrator.py:156 (run), :172 (auto_approve guard), :295 (Sandbox test)`
Impact measurement + rollback for applied improvements	production	`packages/bootstrap/src/animus_bootstrap/intelligence/tools/builtin/self_improve.py:321 (_measure_impact), :387 (_compute_impact_score), packages/bootstrap/src/animus_bootstrap/intelligence/tools/builtin/sandbox.py:140 (rollback)`
Bootstrap runtime orchestrator wiring the loop	production	`packages/bootstrap/src/animus_bootstrap/runtime.py:784 (_create_proactive_engine), :243 (set_self_heal_deps wiring), :316 (config reload hook)`

Feature	Maturity	Evidence
YAML workflow evolution fast-path (documented, partial)	experimental	`docs/WORKFLOW_EVOLUTION_CONSTRAINTS.md:27 (validate_workflow_patch), :84 (staging pattern)`

Observed limitations: The README/quickstart YAML 'gates:' block (check: quality_score >= 0.8) is NOT parsed by the production Forge loader: WorkflowConfig (loader.py:219) has name/version/steps/inputs/outputs/token_budget/timeout/settings but no 'gates' field. Quality-gate enforcement only exists in the separate Core embedded engine (gates.py); the production executor's only gating primitive is the 'approval' step type.; BudgetManager.allocate() (manager.py:264) does not actually reserve or track a pending allocation, it returns True after can_allocate and the comment says 'not yet recorded as used', so concurrent/parallel steps can each pass can_allocate and collectively overspend before record_usage runs.; effective_tokens is computed and exposed (total_effective_tokens / per-agent), but the executor's budget enforcement path (_check_budget_exceeded / record_usage at executor_core.py:255) still uses raw rolled-up tokens, not Effective-Tokens, so the cost-weighted model is reporting-only, not enforced.; Per-step budget check uses a static step.params['estimated_tokens'] default of 1000 (executor_core.py:218); there is no feedback from PreflightValidator's richer per-agent estimates into the live step gate.; The YAML evolution fast-path (WORKFLOW_EVOLUTION_CONSTRAINTS.md) is largely specification: validate_workflow_patch and the 'animus evolve' CLID are described in docs but I did not find a corresponding implemented validator module under forge/workflow; treat as experimental.; self_heal tracks already-proposed areas in a module-global set (_proposed_areas) that is process-lifetime only and not persisted, a daemon restart can re-propose the same failing tool, and dedup is per-process not per-store.; reload_config reaches into private ProactiveEngine attributes (runtime.py:340 sets engine._quiet_start/_quiet_end directly) rather than a public setter, coupling the runtime to engine internals.; _compute_impact_score only weighs failure_rate (60) and latency (40); it has no signal for correctness/quality regressions a config change might introduce, so a change that lowers latency while degrading output quality scores positively.; estimate_cost in BudgetManager uses hardcoded 'late 2024' Claude-3 / GPT-4 pricing tables (manager.py:446) that are stale relative to the model_multipliers table, giving two inconsistent cost views.

Vision, Use-Cases & Stated Roadmap

This subsystem is the narrative and intent layer of Animus: the positioning documents, philosophy, use-case catalog, case study, competitive analysis, and the multiple stated roadmaps that declare where the project is headed. It exists to frame Animus as a "personal AI exocortex for cognitive sovereignty", a single-engineer system applying Toyota Production System discipline (visible cost, quality gates, checkpoint/resume, poka-yoke) to multi-agent AI. Its job is to map why Animus is built and for whom, not how the code works.

Feature	Maturity	Evidence
Three/Four-Layer Architecture Narrative (Core / Forge / Quorum / Bootstrap)	production	`docs/WHITEPAPER.md:30-66; README.md:42-65; CLAUDE.md (Layer Overview, Monorepo Structure)`
TPS / Lean-Manufacturing Framing (budget-first, make cost visible, Jidoka/Poka-yoke/Andon)	production	`docs/WHITEPAPER.md:158-173; docs/EVOLUTION_LOOP.md (TPS Mapping table); CLAUDE.md Non-Negotiable #1 (BudgetManager); README.md:11,250`
Stigmergic Coordination Pitch (no supervisor, intent graph, stability scoring)	production	`docs/WHITEPAPER.md:208-278; animus-landscape-and-additional-tools.md:119-120; CLAUDE.md (Quorum layer); README.md:49-53`
Competitive Landscape & Differentiation Analysis	production	`docs/animus-landscape-and-additional-tools.md:1-149; docs/OPENCLAW_COMPARISON.md:1-57`
Local-Default / Cloud-on-Consent Sovereignty Positioning	production	`docs/OPENCLAW_COMPARISON.md:43-52; docs/specs/openrouter-provider.md referenced; CLAUDE.md Constitutional P1 (Sovereignty)`
Constitutional Principles (P1-P9) as Stated Ethics Frame	production	`docs/CONSTITUTIONAL_PRINCIPLES.md:1-33; CLAUDE.md Non-Negotiable #8`
Personal Roadmap, Single-User Care-and-Feeding Doctrine	production	`docs/PERSONAL_ROADMAP.md:9-264`
Phased Implementation Roadmap (Phase 0-6)	beta	`docs/ROADMAP.md:31-489; contrast CASE_STUDY.md:133 (Phase 5 ✅) vs ROADMAP.md:334-340 (all unchecked)`
Research-Assistant Roadmap (RA-0 through RA-4)	beta	`docs/ROADMAP_research_assistant.md:1-202; packages/forge/src/animus_forge/agent/spec.md referenced`
	beta	`docs/ROADMAP_quorum_v2.md:1-30 (Out of scope table); PERSONAL_ROADMAP.md:162-179 (Track 8)`

Feature	Maturity	Evidence
Quorum v2 Extension Roadmap (selective, with explicit rejections)		
Use-Case Catalog (personal-assistant scenarios)	experimental	`docs/USE_CASES.md:1-330`
Case Study (portfolio/interview artifact)	experimental	`docs/CASE_STUDY.md:11-21,208-266; contrast README.md:244 (13,676 tests)`
Developer-Tools / Arete-Tools Ecosystem Vision (gateway product strategy)	stub	`docs/WHITEPAPER.md:329-341; animus-landscape-and-additional-tools.md:152-735 (Part 4-5 tool specs); contrast PERSONAL_ROADMAP.md:209-228`
Media Engine flagship 'production deployment' (~480 videos/month)	stub	`docs/WHITEPAPER.md:282-326 vs verified absence: grep for story_fire/media_engine/marketing_engine returns only docs and .venv; packages/forge/workflows/ contains only dev/fleet-ops YAML (code-review, fleet-incident-triage, etc.)`

Observed limitations: Flagship deployment is fictional in-repo: the whitepaper's ~480-videos/month Media Engine and 5-platform Marketing Engine (the primary 'this architecture handles real workloads' proof) have zero implementing code. grep finds only vendored googleapiclient stubs and an unrelated Lugh digest. The actual workflows/ are developer/fleet-ops oriented (code-review, fleet-incident-triage, security-audit, bounty-watcher).; Documents contradict each other on maturity and metrics: CASE_STUDY.md claims Phase 5 self-learning 'Complete' and 333 tests/18K LOC, while ROADMAP.md shows all Phase 5 tasks unchecked and README.md reports 13,676 tests. The case study was never updated after build-out.; Strategic whiplash is undocumented as a pivot: the WHITEPAPER + landscape doc sell a commercial gateway/PyPI-suite product strategy (revenue tiers, $8-49/mo), while the PERSONAL_ROADMAP explicitly forbids productization (no SSO/RBAC/billing/landing-page/external-docs). A reader handed both has no signpost that the latter supersedes the former.; Multiple parallel, partially-overlapping roadmaps (ROADMAP.md, PERSONAL_ROADMAP.md, ROADMAP_research_assistant.md, ROADMAP_quorum_v2.md) with no single authoritative index stating which is canonical and which is historical.; The 9-tool 'Arete Tools' suite (Signal, Autopsy, Verdict, Context-Hygiene, Prompt-Debt, Agentlint, Calibrate, Provenance, Tenure) is presented with detailed specs, prompts, and revenue models but is overwhelmingly 'spec complete' / 'this doc', no in-repo implementation; the failure-taxonomy these tools depend on (goal_necrosis, tool_hallucination, etc.) is referenced as if it exists ('Autopsy taxonomy') but is unproven in this repo.; Active-inference IntentResolver, the 'single behavior change worth making' across Quorum v2 and PERSONAL_ROADMAP Track 8, is not yet in code (grep for active_inference/surprise in packages/quorum returns nothing); it remains in-flight roadmap.; Phase 6 (wearable ring, vehicle/CarPlay integration, portable-storage device mode) and the use-cases that depend on it (vehicle mode, location-aware ambient, portable storage) are pure aspiration with no hardware or code path.; Use-case catalog and many case-study capabilities (cross-device WebSocket+Zeroconf sync, register translation, learning-system approval flow) are presented as system capabilities but several are roadmap-checkbox-incomplete (e.g., ROADMAP register translation unchecked; sync layer unchecked).